

Compositional Spatial Planning

by

Yichuan Song

CAES9542 Full Progress Report
BEng(CompSc)
(3035844871, songyc@connect.hku.hk)
The University of Hong Kong
2025

Supervisor:

Dr Lingpeng Kong
Asst.Prof. in Computer Science Dept.

ABSTRACT

In the Robotics field, error accumulation has long been an issue for long-horizon planning tasks such as robot manipulation and autonomous driving. Such issue limits the capacity of existing methods to and has been actively studied. However, most current methods are either domain-specific and cannot generalize to out-of-distribution tasks, or they augment hallucinated pretrained planners with traditional search algorithms which could produce huge computational overheads. This project tackles this problem by providing an energy-based perspective. Specifically, we propose *Compositional Spatial Planning* (CSP) — a novel training-and-inference paradigm that captures multiple dependencies (modality, temporal consistency, and goal-achievement) within the planning trajectory by modeling several energy landscapes through diffusion-style noise propagation, and using the composed landscape to iteratively decode the planning trajectory. We will demonstrate our paradigm’s superiority on spatial-planning tasks of increasing complexity, showing reduced cumulative error, improved performance, and better generalization compared to baseline approaches.

ACKNOWLEDGEMENTS

Thanks to Mr. Jiazhi Yang, Dr. Xiangyu Yue, and Dr. Yilun Du for their advice, and to Prof. Xiangyu Yue for providing the computational resources.

TABLE OF CONTENTS

ABSTRACT	ii
ACKNOWLEDGEMENTS	iii
LIST OF FIGURES	vi
LIST OF TABLES	vii
LIST OF ACRONYMS	viii
CHAPTER	
1 Introduction	1
2 Related Works	2
2.1 Long-Horizon Planning and Error Accumulation	2
2.2 Energy-Based Models and Applications in Planning	3
3 Compositional Spatial Planning	4
3.1 Problem Formulation	4
3.2 Mapping Data into Latent Space	4
3.3 Three Types of Dependencies	4
3.4 Disjointly Training with Noise Propagation	6
3.5 Sampling with Iterative Refinement	6
4 Experiments	7
4.1 Paradigm Validation Stage	7
4.1.1 Fully-Observable Environments	7
4.1.2 Experiment Settings	8
4.1.3 Current Progress and Analysis	8
4.2 Task Generalization Stage	9
4.3 Multi-modal Alignment Stage	9
5 Conclusion	10
APPENDICES	11

BIBLIOGRAPHY	16
------------------------	----

LIST OF FIGURES

FIGURE

3.1	An overview of CSP training – CSP captures three kinds of latent dependencies- multi-modalities(pink), temporal consistency(blue) and goal achievement(green)-with three energy landscapes, correspondingly.	5
4.1	Stage 1 Tasks. – Stage 1 tasks in fully observable environments with increasing DoF (left to right). Particle is trivial; Reacher and Sawyer require both vision and motion modalities.	8
B.1	A flowchart of CSP sampling – Different EBMs with their input-output flows are represented in different colors: the green ones are for goal-achievement, the pink ones are for multi-modality and the blue ones are for temporal-consistency. For inference, we incrementally expand the context observables and incorporates the EBMs for sampling, with order: $E_{\theta_1} \rightarrow E_{\theta_1 \cup \theta_2} \rightarrow E_{\theta_1 \cup \theta_2 \cup \theta_3}$	15

LIST OF TABLES

TABLE

4.1	Task Relationships. – We evaluate CSP across a spectrum from trivial short-horizon planning to language-instructed long-horizon planning.	9
-----	--	---

LIST OF ACRONYMS

BERT Bidirectional Encoder Representations from Transformers

CoT Chain of Thoughts

CSP Compositional Spatial Planning

DoF Degree-of-Freedom

EBM Energy-Based Model

FYP Final Year Project

LRM Language Reasoning Model

MCMC Markov Chain Monte Carlo

OOD Out-Of-Domain

RAE Representation Auto-Encoder

SVGD Stein Variational Gradient Descent

VAE Variational Auto-Encoder

VLA Vision-Language-Action Model

VLM Video-Language Model

VM Video Model

CHAPTER 1

Introduction

Long-horizon planning remains a cornerstone of robotics[22, 28, 7, 27], yet prevailing paradigms still stumble over compounding errors: language-backed planners hallucinate[32, 19], video simulators drift frame-by-frame[12, 5, 9], and hierarchical pipelines burn compute in endless re-simulation[13, 11, 33]. Giving these drawbacks of the existing methods, a natural question to ask would be *whether error accumulation in long-horizon planning can be mitigated by directly modeling global and local dependencies*. We therefore introduce *Compositional Spatial Planning* (CSP), an energy-based trajectory planning paradigm in the Robotics field, aiming to capture different levels of dependencies using corresponding energy landscapes. During learning, CSP explicitly shapes energy landscapes of multi-modal, temporal and goal-achieving dependencies from the trajectories of frames and motions. At inference time, these landscapes are composed together and iteratively guide the denoising process of both trajectories of frames and motions with their gradients. To make the noise propagation tractable and more computationally efficient, CSP adapts modality-specific auto-encoders to map visual frames, textual queries and robot motions to their corresponding representations in the latent spaces. We aim to demonstrate the superiority of CSP on spatial-planning tasks of diverse complexities compared with current baselines, exhibiting reduced error accumulation, higher goal-achieving rates, scalable test-time computation, and better generalization to complex and out-of-domain tasks. Deliverables will include a conference submission, open-source code, and checkpoints.

CHAPTER 2

Related Works

2.1 Long-Horizon Planning and Error Accumulation

Long-horizon planning has been a cornerstone problem in robotics, autonomous driving and task-oriented RL. Early work relied on classical search (A*, FF, PDDL-based planners [22]) or hierarchical abstractions such as options [28] and MAXQ [7] to reduce depth; however, these methods assume short-horizon or discrete action spaces and struggle with high-dimensional observations. With the rise of deep generative models, two families of approaches dominate recent literature.

Video-predictive dynamics treat planning as iterative frame prediction [24, 12, 5]; while accurate for a few tens of steps, error variance grows quadratically [10], leading to unrealistic frames and control divergence. Hierarchical remedies decompose tasks into sub-goals [23] or leverage contrastive objectives [27] to regularise latent space, yet still require explicit short-rollout simulation and expensive re-planning.

Language-backed planners embed observations into text tokens and exploit large language or vision-language models (LMs/VLMs) as high-level policies [26, 17, 13, 3]. Chain-of-thought prompting [32] and code-as-policies [19] achieve impressive logical consistency over dozens of steps, but inherit LM hallucinations and lack fine-grained spatial or dynamic fidelity. Compositional pipelines that interleave VLMs, VLAs and VMs inside MCTS or MPPI [9, 11, 33] alleviate drift by external search, yet incur heavy compute from massive roll-out generation and cross-modal scoring.

In summary, existing paradigms either accumulate predictive error (video dynamics) or semantic hallucination (language planners), and compensate by shallow re-planning rather than global trajectory regularization. Our work differs by directly modeling global-local dependencies within a composed energy landscape, avoiding per-step re-simulation while remaining search-free at test time.

2.2 Energy-Based Models and Applications in Planning

Energy-Based Models (EBMs) represent data dependency through an unnormalized energy function, enabling flexible composition of constraints and modalities without explicit probabilistic modeling. Early planning applications used Boltzmann machines for discrete action selection [29] and potential fields for robot motion [15]. The modern revival began with Joint-Embedding Predictive Architecture (JEPA) [18] and contrastive predictive coding, where EBMs are trained by noise-contrastive or score-matching objectives. In control, Score-Flow [14] and Diffusion-QL [31] show that diffusion-style EBMs can generate smooth, globally coherent trajectories for offline RL and manipulation.

Recent work extends EBMs to multi-modal planning: Decision Diffuser [1] combines return, constraint and skill energy terms; Compositional Diffusion [14] demonstrates Boolean-like composition (AND/OR/NOT) of energy landscapes for multi-task robotics. However, these methods still rely on autoregressive denoising and short-horizon reconstruction, hence error accumulation re-emerges when roll-outs exceed 200 steps.

Our *Compositional Spatial Planning* (CSP) paradigm advances the energy-based planning literature in three ways: (1) compositional local-global trajectory energy instead of monotonic frame-wise or token-wise score, (2) latent-space diffusion that propagates noise along the entire horizon, satisfies mid-term constraints, and exhibits bounded error growth, and (3) iterative gradient-based decoding that does not require re-simulation or expensive search at test time. Consequently, CSP retains the compositional flexibility of EBMs while eliminating the drift that plagues prior diffusion planners.

CHAPTER 3

Compositional Spatial Planning

In this section, we introduce our CSP paradigm in detail. After formulating the problem setting in 3.1, we present the encoders and decoders that convert high-dimensional, multi-modal raw data into aligned latent representations in 3.2. Next, we define and model the three key dependencies—modality, temporal consistency, and goal achievement—in 3.3. With these components in place, we construct our diffusion-style training and iterative sampling pipelines in 3.4 and 3.5, correspondingly. Complete pseudocodes for training and sampling are provided in Appendix A.

3.1 Problem Formulation

Given a textual instruction $\mathbf{g}$ that describes the planning goal and an initial state represented by the image frame $\mathbf{x}_0$ at time $t = 0$, we aim to generate a planning trajectory $\{\mathbf{a}_1, \mathbf{x}_1, \mathbf{a}_2, \mathbf{x}_2, \dots, \mathbf{a}_t, \mathbf{x}_t\}$ starting from $\mathbf{x}_0$ to achieve $\mathbf{g}$. The trajectory consists of an alternating sequence of actions and resulting frames, where each action $\mathbf{a}_i$ transitions the system from frame $\mathbf{x}_{i-1}$ to frame $\mathbf{x}_i$, and the final frame $\mathbf{x}_t$ is required to satisfy the goal $\mathbf{g}$.

3.2 Mapping Data into Latent Space

(Due to the page limitation specified by the CAES9542 guideline, we move this section to Appendix B.1)

3.3 Three Types of Dependencies

By intuition, we want to capture both local and global planning dependencies by modeling distinct energy functions that operate on latents representing diverse modalities, temporal consistency, and goal achievement. Specifically, we define three energy functions to capture the corresponding dependencies:

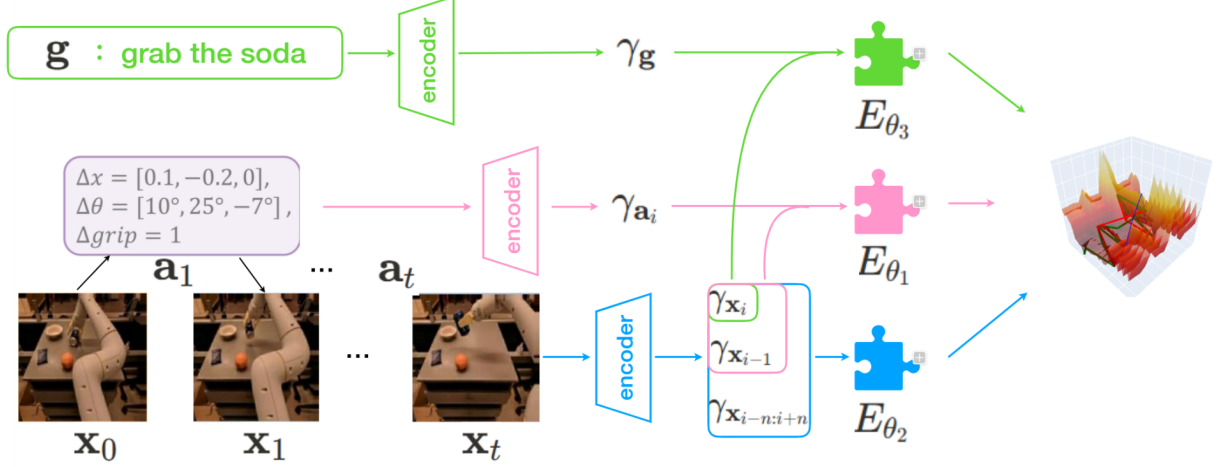


Figure 3.1: **An overview of CSP training** – CSP captures three kinds of latent dependencies—multi-modalities(pink), temporal consistency(blue) and goal achievement(green)—with three energy landscapes, correspondingly.

Goal-achieving dependency. The first dependency aims to capture the global planning requirement:

$$E_{\theta_1}(\gamma_{x_0}, \gamma_{x_i}, \gamma_g) : \mathbb{R}^b \times \mathbb{R}^b \times \mathbb{R}^a \rightarrow \mathbb{R}, i = 1, \dots, t \quad (3.1)$$

As shown in Figure 3.1, the goal-achieving dependency is modeled by the landscape E_{θ_1} (colored in green), which takes the three latent inputs $\gamma_{x_0}, \gamma_{x_i}, \gamma_g$ and returns the corresponding scalar energy. Given the initial state γ_{x_0} and the goal γ_g , the trained landscape is expected to assign the lowest energy to the optimal current frame γ_{x_i} in the latent space: $\gamma_{x_i}^* = \arg \min_{\gamma_i} E_{\theta_1}^*(\gamma_{x_0}, \gamma_i, \gamma_g)$.

Intuitively, E_{θ_1} measures how well the current frame achieves the specified goal given the initial state in the latent space, with lower energy indicating better correspondence. Since γ_{x_0} and γ_g are fixed, we only need to model the landscape with respect to x_i .

Multi-modal dependency. The second dependency measures the relationship between different modalities at the same time step and serves as a local planning constraint:

$$E_{\theta_2}(\gamma_{x_{i-1}}, \gamma_{x_i}, \gamma_{a_i}) : \mathbb{R}^b \times \mathbb{R}^b \times \mathbb{R}^c \rightarrow \mathbb{R}, i = 1, \dots, t \quad (3.2)$$

As shown in Figure 3.1, the modality dependency is modeled by the landscape E_{θ_2} (colored in pink), which takes three latent inputs: the current motion γ_{a_i} and the consecutive frames $\gamma_{x_{i-1}}$ and γ_{x_i} before and after the current motion. Since the $\gamma_{x_{i-1}}$ and γ_{x_i} are fixed during both training and sampling, we only need shape this energy landscape with respect to γ_{a_i} : $\gamma_{a_i}^* = \arg \min_{\gamma_i} E_{\theta_2}^*(\gamma_{x_{i-1}}, \gamma_{x_i}, \gamma_i)$.

Temporal dependency. The final dependency measures temporal consistency within a predefined context window of $2n$ frames, capturing the regional planning dependency at the current time step:

$$E_{\theta_3}(\gamma_{\mathbf{x}_{i-n:i+n}}) : \mathbb{R}^{b \times 2n} \rightarrow \mathbb{R}, i = n, \dots, t - n \quad (3.3)$$

As shown in Figure 3.1, temporal consistency is modeled by the landscape E_{θ_3} (colored in blue), which takes the frame sequence $\gamma_{\mathbf{x}_{i-n:i+n}} \{\gamma_{x_{i-n}}, \dots, \gamma_{x_i}, \dots, \gamma_{x_{i+n}}\}$ as input and outputs a scalar energy representing regional consistency.

Unlike the previous two energy functions, the E_{θ_3} landscape is shaped with respect to all input latents, giving it the highest complexity for training and sampling. Nevertheless, this EBM can still be learned and used for inference with efficient methods, which we leave for future work.

Compositional landscape. Ultimately, we simply sum the three energy landscapes to obtain the final composed landscape:

$$E_{\theta} = E_{\theta_1} + E_{\theta_2} + E_{\theta_3}, \text{ where } \theta = \theta_1 \cup \theta_2 \cup \theta_3 \quad (3.4)$$

3.4 Disjointly Training with Noise Propagation

(Due to the page limitation specified by the CAES9542 guideline, we move this section to Appendix B.2)

3.5 Sampling with Iterative Refinement

(Due to the page limitation specified by the CAES9542 guideline, we move this section to Appendix B.3)

CHAPTER 4

Experiments

To validate and examine the superiority of our paradigm, we designed a series of planning tasks of increasing complexity, divided into three stages. In 4.1, we evaluate the video-motion planning capability of the CSP paradigm in simple, fully observable environments; at the time of writing, all reported progress falls within this first stage (see 4.1.3). 4.2 and 4.3 briefly outline the two subsequent stages planned for future work.

Table 4.1 shows the relationship of all the tasks we plan to examine our CSP with.

4.1 Paradigm Validation Stage

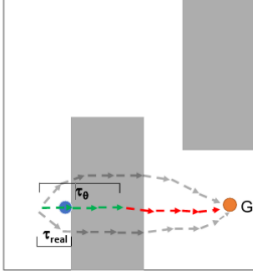
The goal of this first stage is to verify the feasibility and effectiveness of CSP in a straightforward setting. All tasks are adapted from [9]. 4.1.1 describes three fully observable environments in which the model must generate a complete frame-motion trajectory in a single step. Technical details are provided in 4.1.2, and preliminary results are presented and analyzed in 4.1.3. We expect to finish this stage within approximately one month by the end of 2025.

4.1.1 Fully-Observable Environments

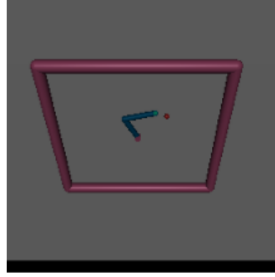
To better suit our testing goal, we change the original online-learning setting to an offline trajectory-prediction task in which both global states and motions must be generated.

1. Particle Environment. A particle is spawned at a start position and must navigate to a goal position. The environment state is fully observable, including the (x,y) coordinates of the start and goal points and the forbidden regions. The particle state is its current (x,y) position, so the full state is the union of the environment and particle states. The motion is the x- and y-displacement (two degrees of freedom) of the particle.

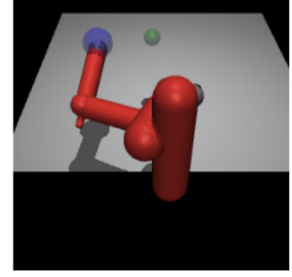
2. Reacher Environment. A classic continuous-control task from [2]. The “Reacher” is a two-jointed robot arm whose end-effector (fingertip) must reach a randomly spawned



Particle Environment



Reacher Environment



Sawyer Environment

Figure 4.1: **Stage 1 Tasks.** – Stage 1 tasks in fully observable environments with increasing DoF (left to right). Particle is trivial; Reacher and Sawyer require both vision and motion modalities.

target. The motion vector (a,b) commands the angular velocities of the two joints, and the state comprises the initial and target end-effector positions together with the current joint angles and angular velocities.

3. Sawyer Environment. A more complex 7-degree-of-freedom equivalent from [30]. The objective is identical: move the robot-arm end-effector to the target position.

4.1.2 Experiment Settings

Originally designed for online learning, the three tasks are converted to offline trajectory prediction. We record 1,000 videos (3 fps, 128×128 RGB) per environment with random start/goal states and an 8:2 success-to-failure ratio; 100 additional videos are held out for testing. Except for the Particle domain, both Reacher and Sawyer provide vision and motion modalities; for these we train ViT-based VAE encoders/decoders [8, 16] from scratch, optimizing end-to-end with AdamW[21] (learning rate at $1e^{-3}$, 10 epochs).

4.1.3 Current Progress and Analysis

As of submission we are validating the paradigm on the Particle environment, having (i) surveyed planning tasks for all three stages, (ii) collected 1,000 video-motion trajectories (20 steps each), and (iii) built a modular codebase for data, training, inference and visualisation. Training is still in progress, so no quantitative results are available yet; below we summarise anticipated bottlenecks and remedies.

Dimensionality. Low-dimensional states suffice for Stage 1, but Stages 2–3 require high-resolution video. We compress modalities with modality-specific VAE/RAE encoders[16, 34]: ViT[8] for frames motion, BERT[6] for text, keeping memory and time tractable.

Sampling cost. Naïve full-sequence denoising of CSP is autoregressive and slow. We will adopt divide-and-conquer interpolation (sample key frames every N/M steps, denoise

Stage	Task	DoF	Modality			State Observability	Status
			Vision	Motion	Language		
1	Particle	2	-	-	-	Full	In progress By Dec 31st
	Reacher	2	✓	✓		Full	
	Sawyer	7	✓	✓		Full	
2	AntMaze	8	✓	✓		Partial	By Jan 12th
	AntSoccer	10	✓	✓		Partial	
3	Assemble Interlocked Pieces	10	✓	✓	✓	Partial	By Apr 19th
	Make a Line	10	✓	✓	✓	Partial	
	Put Fruits in Drawer	10	✓	✓	✓	Partial	

Table 4.1: **Task Relationships.** – We evaluate CSP across a spectrum from trivial short-horizon planning to language-instructed long-horizon planning.

chunks in parallel) and accelerate mixing with Stein variational[20] or Hamiltonian-guided MCMC[4].

4.2 Task Generalization Stage

We will deploy CSP in OGBench AntMaze and AntSoccer[25], where a quadruped must escape a 3-D maze or push a ball to goal. After offline preprocessing of success/failure trajectories, we will upgrade the energy-function architectures to handle 3-D joint space and textured RGB. Completion is targeted before the mid-January FYP presentation.

4.3 Multi-modal Alignment Stage

Finally, we introduce textual variability: CSP will solve language-instructed tasks (Assembling-Interlocked-Pieces, Making-a-Line, Putting-Fruits-in-the-Top-Drawer) from [9, 11]. All experiments and ablations will finish before the final FYP presentation in late April 2026.

CHAPTER 5

Conclusion

In this report, we introduced a novel learning-and-inference paradigm called *Compositional Spatial Planning*, or CSP, for long-horizon planning tasks in robotics. By modeling various trajectory dependencies with corresponding energy landscapes, CSP is expected to achieve higher success rates, better generalization to out-of-domain tasks, and scalable test-time performance compared with mainstream pretrained and compositional baselines. We have now collected and pre-processed the training and testing data for the Particle environment and implemented the CSP codebase for the first experimental stage, which aims to validate our paradigm. Although no numerical results have been yielded yet, we found that due to the trivial environment setting, we can conduct the experiments of the paradigm validation stage in a considerably efficient way. We also discussed potential obstacles regarding latent encoding and sampling efficiency and proposed corresponding solutions.

Our contributions are manifold: (1) We present CSP, a new energy-based approach that mitigates the error-accumulation issue prevalent in existing planning methods. (2) Owing to the flexibility and expressiveness of EBMs, our method is a step toward real-world generalizability. (3) We provide validated alternatives for latent encoding and efficient sampling that may benefit research fields beyond robotics and encourage theoretical advances through domain-specific experimental results.

Nevertheless, the current paradigm cannot yet be deployed directly in real-world planning because of information loss during latent encoding and the computational cost of sampling. We have outlined several countermeasures and will evaluate their feasibility in the next stage. Future work can be divided into modeling and sampling aspects. For modeling, instead of explicitly shaping an energy landscape for each dependency, one could leverage pretrained models to compose an implicit target distribution for test-time sampling without additional parameter tuning. For sampling, principles from statistical mechanics and advanced MCMC techniques could accelerate the process. We encourage further exploration of these directions.

APPENDIX A

Training and Sampling Algorithms

A.1 CSP Training

Algorithm 1 CSP Training

Require: train dataset $\mathcal{D}$, encoders f_x, f_a, f_g , EBMs $E_{\theta_1}, E_{\theta_2}, E_{\theta_3}$, sample frames t , noise schedules $\{\sigma_k\}$, diffusion timestep K , window size $2n$

- 1: **while** not converged **do**
- 2: $(\mathbf{g}, \mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_t, \mathbf{a}_1, \dots, \mathbf{a}_t) \sim \mathcal{D}, \epsilon \sim \mathcal{N}(0, 1), k \sim \{1, \dots, K\}$
 Encode the raw data into latent space
- 3: $\gamma_g \leftarrow f_g(\mathbf{g}), \gamma_{\mathbf{x}0:t} \leftarrow f_x(\mathbf{x}_{0:t}), \gamma_{\mathbf{a}1:t} \leftarrow f_a(\mathbf{a}_{1:t})$
- 4: **for** $i = 1$ to t **do**
 Add noise to the latent inputs
- 5: $\tilde{\gamma}_{\mathbf{x}i} \leftarrow \sqrt{1 - \sigma_k^2} \gamma_{\mathbf{x}i} + \sigma_k \epsilon$
 Calculate the modal correlation, temporal correlaton, and goal-achievement
- 6: $\mathcal{E}_i^{\text{mod}} \leftarrow E_{\theta_1}(\tilde{\gamma}_{\mathbf{x}i}, \gamma_{\mathbf{x}i-1}, \gamma_{\mathbf{a}i})$
- 7: $\mathcal{E}_i^{\text{temp}} \leftarrow E_{\theta_2}(\gamma_{\mathbf{x}i-n:i+n}, \gamma_{\mathbf{a}i}), \quad i + n < t$
- 8: $\mathcal{E}_i^{\text{goal}} \leftarrow E_{\theta_3}(\gamma_{\mathbf{x}i}, \gamma_g)$
 Train with the compositional energy objective
- 9: $\mathcal{E}_i \leftarrow \mathcal{E}_i^{\text{mod}} + \mathcal{E}_i^{\text{temp}} + \mathcal{E}_i^{\text{goal}}$
- 10: $\Delta\theta_1 \leftarrow \nabla_{\theta_1}(\|\nabla_{\gamma_{\mathbf{x}i}} \mathcal{E}_i - \epsilon\|^2)$
- 11: $\Delta\theta_2 \leftarrow \nabla_{\theta_2}(\|\nabla_{\gamma_{\mathbf{x}i-n:i+n}} \mathcal{E}_i - \epsilon\|^2)$
- 12: $\Delta\theta_3 \leftarrow \nabla_{\theta_3}(\|\nabla_{\gamma_{\mathbf{x}i}, \gamma_g} \mathcal{E}_i - \epsilon\|^2)$
- 13: **end for**
- 14: Update $\theta_1, \theta_2, \theta_3$ on $\Delta\theta_1, \Delta\theta_2, \Delta\theta_3$ using Adam optimizer
- 15: **end while**

A.2 CSP Sampling

Algorithm 2 CSP Sampling

```

1: Models: trained encoders  $f_{\text{vision}}, f_{\text{goal}}, f_{\text{motion}}$ ; trained decoders  $g_{\text{vision}}, g_{\text{motion}}$ ; trained EBMs
    $E_{\theta_1}(\gamma_{\mathbf{x}0}, \gamma_{\mathbf{x}i}, \gamma_g), E_{\theta_2}(\gamma_{\mathbf{x}i-1:i}, \gamma_{\mathbf{a}i}), E_{\theta_3}(\gamma_{\mathbf{x}i-n:i+n})$ 
2: Input: initial frame  $\mathbf{x}_0$ , goal  $\mathbf{g}$ , frames  $N$ , gram size  $2n$ , sampling steps  $T$ 
3: Initialize latent buffers  $B_{\mathbf{x}}, B_{\mathbf{a}}$ 
4: Initialize compositional energy  $\mathcal{E} \leftarrow \infty$ 
5: Initialize latents  $\gamma_{\mathbf{x}1:N}, \gamma_{\mathbf{a}1:N} \sim \mathcal{N}(0, I)$ 
6:  $\gamma_g \leftarrow f_{\text{goal}}(\mathbf{g}), \gamma_{\mathbf{x}0} \leftarrow f_{\text{vision}}(\mathbf{x}_0)$ 
7: for  $t = 1$  to  $T$  do
8:   for  $i = 1$  to  $N$  do
9:     first: denoise frame latent at  $i$ 
10:     $\gamma'_{\mathbf{x}i} \leftarrow \gamma_{\mathbf{x}i} - \nabla_{\gamma_{\mathbf{x}i}} E_{\theta_1}(\gamma_{\mathbf{x}0}, \gamma_{\mathbf{x}i}, \gamma_g)$ 
11:    second: denoise motion latent at  $i$ 
12:     $\gamma'_{\mathbf{a}i} \leftarrow \gamma_{\mathbf{a}i} - \nabla_{\gamma_{\mathbf{a}i}} E_{\theta_1+\theta_2}(\gamma_{\mathbf{x}i-1}, \gamma_{\mathbf{x}i}, \gamma_{\mathbf{a}i}, \gamma_g)$ 
13:    third: joint update of frame latent at  $i$ 
14:    if  $n \leq i \leq N - n$  then
15:       $\gamma''_{\mathbf{x}i} \leftarrow \gamma'_{\mathbf{x}i} - \nabla_{\gamma'_{\mathbf{x}i}} E_{\theta_1+\theta_2+\theta_3}(\gamma'_{\mathbf{x}i-n:i+n}, \gamma'_{\mathbf{a}i-n:i+n}, \gamma_g)$ 
16:       $\gamma''_{\mathbf{a}i} \leftarrow \gamma'_{\mathbf{a}i} - \nabla_{\gamma'_{\mathbf{a}i}} E_{\theta_1+\theta_2+\theta_3}(\gamma''_{\mathbf{x}i-n:i+n}, \gamma'_{\mathbf{a}i-n:i+n}, \gamma_g)$ 
17:      if  $E_{\theta}(\gamma_{\mathbf{x}0:n}, \gamma''_{\mathbf{x}i-n:i+n}, \gamma_g) < E_{\theta}(\gamma_{\mathbf{x}0:n}, \gamma_{\mathbf{x}i-n:i+n}, \gamma_g)$  then
18:         $\gamma_{\mathbf{x}i-n:i+n} \leftarrow \gamma''_{\mathbf{x}i-n:i+n}, \gamma_{\mathbf{a}i-n:i+n} \leftarrow \gamma''_{\mathbf{a}i-n:i+n}$ 
19:      end if
20:    end if
21:  end for
22: end for
23: return  $\mathbf{x}_1, \dots, \mathbf{x}_N, \mathbf{a}_1, \dots, \mathbf{a}_N$ 

```

APPENDIX B

Supplementary Details of CSP

B.1 Mapping Data into Latent Space

For computational efficiency, we model the energy landscapes on the latent representations of the different modalities, instead of directly modeling on the raw data of high-dimensions. For the choice of design, this paper considers three different modalities: textual planning instruction $\mathbf{g}$, vision frame $\mathbf{x}$, and motion $\mathbf{a}$.¹ Due to the different nature of these modalities, we adapt different encoders to map them to the corresponding latent vectors $\gamma_{\mathbf{g}} \in \mathbb{R}^a$, $\gamma_{\mathbf{x}} \in \mathbb{R}^b$ and $\gamma_{\mathbf{a}} \in \mathbb{R}^c$, where a, b and c are the corresponding latent dimensions, and do the reverse process with decoders, correspondingly. Generally, We train our encoders and decoders with the Variational Autoencoder(VAE) [16] setting. Specifically, as for architecture, we choose BERT [6] for processing the discrete textual query, and ViT [8] for processing continuous image frames and robot motions, following the empirical treatments.

B.2 Disjointly Training with Noise Propagation

Following the traditional EBM training paradigm, we train the three EBMs separately using diffusion-style noise propagation. In our implementation, we apply an inverse-cosine noise schedule to the latent vectors on which the three landscapes are shaped—namely γ_{x_i} , γ_{a_i} , and $\gamma_{\mathbf{x}_{i-n:i+n}}$ —and leave alternative schedules for future work. We iteratively add Gaussian noise to each latent; after T iterations, it becomes random Gaussian noise.

¹For each planning task covered in our experiment in 4, the number of different modalities involved is at least two(frame and motion) and at most three(text, frame and motion). However, our paradigm can be easily generalized to involve more modalities, such as audio and other sensors, and we encourage future works to explore in these directions.

$$\begin{aligned}
\tilde{\gamma}^0 &= \gamma \\
\tilde{\gamma}^t &= \tilde{\gamma}^{t-1} + \epsilon, \epsilon \sim \mathcal{N}(0, 1), k = 2, \dots, T \\
\tilde{\gamma}^T &= \text{Gaussian}
\end{aligned} \tag{B.1}$$

We then learn the energy functions $E_{\theta_1}, E_{\theta_2}, E_{\theta_3}$ by supervising their gradients to denoise the noise-corrupted latent at each diffusion step k and time stamp i using mean squared error:

$$\mathcal{L}_{\text{MSE},n}(\theta) = \|\nabla_{\mathbf{y}_n} E_{\theta}(\gamma_{x_0}, \gamma_g, \tilde{\gamma}_{\mathbf{x}_{i-n:i+n}}^k) - \epsilon_k\|^2, n = 1, 2, 3 \tag{B.2}$$

where y_1, y_2 and $\mathbf{y}_3$ represent $\gamma_{x_i}, \gamma_{a_i}$ and $\gamma_{\mathbf{x}_{i-n:i+n}}$, correspondingly.

B.3 Sampling with Iterative Refinement

Given the textual instruction $\mathbf{g}$ and the initial state $\mathbf{x}_0$, we first encode the corresponding latents γ_g and γ_{x_0} using our textual and visual encoders. We then iteratively generate frame and motion latents with the three learned EBMs, and finally decode them into the predicted trajectory $\{\mathbf{a}_1, \mathbf{x}_1, \dots, \mathbf{a}_N, \mathbf{x}_N\}$.

Specifically, after obtaining the goal latent γ_g and the initial-state latent γ_{x_0} , we produce the latent trajectory in a diffusion fashion. Suppose we wish to generate a trajectory $\{\gamma_{a_i}, \gamma_{x_i}\}_{i=1}^N$ of length N over T diffusion steps. At the end of each diffusion step $t = 1, \dots, T$, a complete trajectory $\{\gamma_{a_i}^t, \gamma_{x_i}^t\}_{i=1}^N$ with noise level t is available. Within each diffusion step t , we further denoise and refine the latents near the current time stamp $i = 1, \dots, N$ with an expanding context of observations, as shown in Figure B.1:

First, the goal-achieving EBM E_{θ_1} denoises the current state latent γ_{x_i} conditioned on γ_g and γ_{x_0} (blue circle in Figure B.1) across T' sub-steps:

$$\gamma_{x_i}^{t+1} \leftarrow \gamma_{x_i}^t - \lambda_1 \nabla_{\gamma_{x_i}^t} E_{\theta_1}(\gamma_g, \gamma_{x_0}, \gamma_{x_i}^t) \tag{B.3}$$

where $\lambda_i, i = 1, 2, 3$ are gradient strides for the three EBMs. In our implementation, we initialize the current frame latent with Gaussian noise: $\gamma_{x_i}^0 \sim \mathcal{N}(0, I_x)$

Secondly, the multi-modal EBM E_{θ_2} denoises the latest motion latent γ_{a_i} conditioned on the previous and current frames $\gamma_{x_{i-1}}, \gamma_{x_i}$ over T' sub-steps (green circle in Figure B.1). In our implementation, We combine E_{θ_1} with E_{θ_2} to provide stronger guidance during sampling:

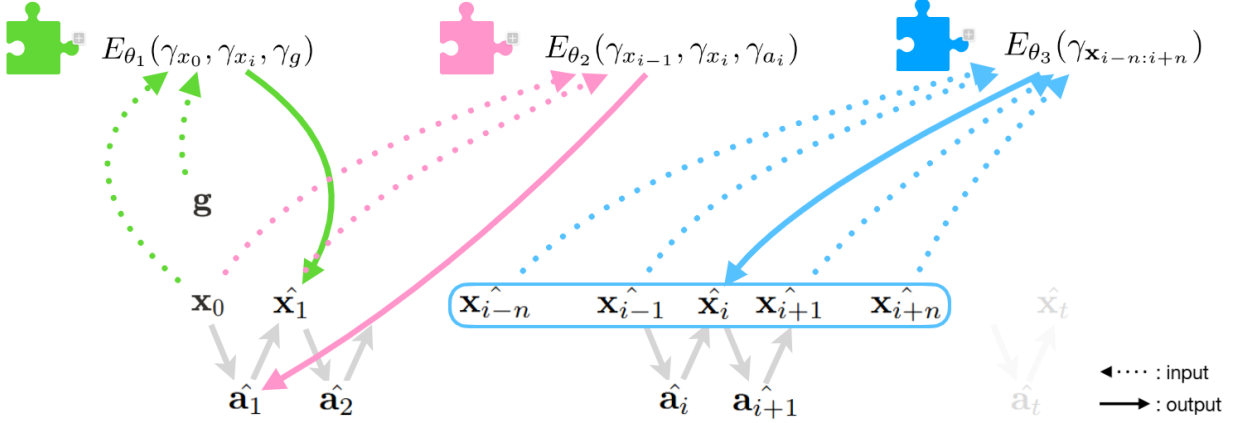


Figure B.1: **A flowchart of CSP sampling** – Different EBMs with their input-output flows are represented in different colors: the green ones are for goal-achievement, the pink ones are for multi-modality and the blue ones are for temporal-consistency. For inference, we incrementally expand the context observables and incorporates the EBMs for sampling, with order: $E_{\theta_1} \rightarrow E_{\theta_1 \cup \theta_2} \rightarrow E_{\theta_1 \cup \theta_2 \cup \theta_3}$.

$$\gamma_{a_i}^{t+1} \leftarrow \gamma_{a_i}^t - \lambda_2 \nabla_{\gamma_{a_i}^t} E_{\theta_1 \cup \theta_2}(\gamma_g, \gamma_{x_0}, \gamma_{\mathbf{x}_{i-1:i}}^t) \quad (\text{B.4})$$

Here we also choose to randomly initialize the motion latent: $\gamma_{a_i}^0 \sim \mathcal{N}(0, I_a)$.

Finally, once the currently denoised latents form a complete context window of size $2n$, the temporal-consistency EBM E_{θ_3} refines each frame latent iteratively, conditioned on the other latents within the window (yellow circle in Figure B.1). As before, our implementation combine E_{θ_3} with the other two EBMs to improve sampling quality:

$$\gamma_{x_j}^{t+1} \leftarrow \gamma_{x_j}^t - \lambda_3 \nabla_{\gamma_{x_j}^t} E_{\theta_1 \cup \theta_2 \cup \theta_3}(\gamma_g, \gamma_{x_0}, \gamma_{\mathbf{x}_{i-n:i+n}}^t) \quad (\text{B.5})$$

where $j = i - n, \dots, i + n$.

After iterating through every time stamp i along the trajectory of length N , CSP produces a complete trajectory at diffusion step t ; after T diffusion steps, it yields the final trajectory prediction.

BIBLIOGRAPHY

- [1] Anurag Ajay, Yilun Du, Abhi Gupta, Joshua Tenenbaum, Tommi Jaakkola, and Pulkit Agrawal. Is conditional generative modeling all you need for decision-making?, 2023.
- [2] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016.
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishk Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023.
- [4] Steve Brooks, Andrew Gelman, Galin Jones, and Xiao-Li Meng. Handbook of Markov Chain Monte Carlo. Chapman and Hall/CRC, May 2011.
- [5] Silvia Chiappa, Sébastien Racaniere, Daan Wierstra, and Shakir Mohamed. Recurrent environment simulators, 2017.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [7] Thomas G. Dietterich. Hierarchical reinforcement learning with the maxq value function decomposition, 1999.
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.
- [9] Yilun Du, Mengjiao Yang, Pete Florence, Fei Xia, Ayzaan Wahid, Brian Ichter, Pierre Sermanet, Tianhe Yu, Pieter Abbeel, Joshua B. Tenenbaum, Leslie Kaelbling, Andy Zeng, and Jonathan Tompson. Video language planning, 2023.

- [10] Vladimir Feinberg, Alvin Wan, Ion Stoica, Michael I. Jordan, Joseph E. Gonzalez, and Sergey Levine. Model-based value estimation for efficient model-free reinforcement learning, 2018.
- [11] Yunhai Feng, Jiaming Han, Zhuoran Yang, Xiangyu Yue, Sergey Levine, and Jianlan Luo. Reflective planning: Vision-language models for multi-stage long-horizon robotic manipulation, 2025.
- [12] Chelsea Finn and Sergey Levine. Deep visual foresight for planning robot motion, 2017.
- [13] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents, 2022.
- [14] Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis, 2022.
- [15] O. Khatib. Real-time obstacle avoidance for manipulators and mobile robots. In Proceedings. 1985 IEEE International Conference on Robotics and Automation, volume 2, pages 500–505, 1985.
- [16] Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2022.
- [17] Julia Kiseleva, Alexey Skrynnik, Artem Zholus, Shrestha Mohanty, Negar Arabzadeh, Marc-Alexandre Côté, Mohammad Aliannejadi, Milagro Teruel, Ziming Li, Mikhail Burtsev, Maartje ter Hoeve, Zoya Volovikova, Aleksandr Panov, Yuxuan Sun, Kavya Srinet, Arthur Szlam, and Ahmed Awadallah. Iglu 2022: Interactive grounded language understanding in a collaborative environment at neurips 2022, 2022.
- [18] Yann LeCun. A path towards autonomous machine intelligence, 2022.
- [19] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control, 2023.
- [20] Qiang Liu and Dilin Wang. Stein variational gradient descent: A general purpose bayesian inference algorithm, 2019.
- [21] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.
- [22] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL—the planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, New Haven, CT, USA, 1998.
- [23] Soroush Nasiriany, Vitchyr H. Pong, Steven Lin, and Sergey Levine. Planning with goal-conditioned policies, 2019.
- [24] Junhyuk Oh, Xiaoxiao Guo, Honglak Lee, Richard Lewis, and Satinder Singh. Action-conditional video prediction using deep networks in atari games, 2015.

- [25] Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. Ogbench: Benchmarking offline goal-conditioned rl, 2025.
- [26] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021.
- [27] Aravind Srinivas, Michael Laskin, and Pieter Abbeel. Curl: Contrastive unsupervised representations for reinforcement learning, 2020.
- [28] Richard S. Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: a framework for temporal abstraction in reinforcement learning. Artif. Intell., 112(1–2):181–211, August 1999.
- [29] Gerald Tesauro. Temporal difference learning and td-gammon. Commun. ACM, 38(3):58–68, March 1995.
- [30] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. pages 5026–5033, 10 2012.
- [31] Zhendong Wang, Jonathan J Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning, 2023.
- [32] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
- [33] Yuncong Yang, Jiageng Liu, Zheyuan Zhang, Siyuan Zhou, Reuben Tan, Jianwei Yang, Yilun Du, and Chuang Gan. Mindjourney: Test-time scaling with world models for spatial reasoning, 2025.
- [34] Boyang Zheng, Nanye Ma, Shengbang Tong, and Saining Xie. Diffusion transformers with representation autoencoders, 2025.